INTERFACES GRAFICAS

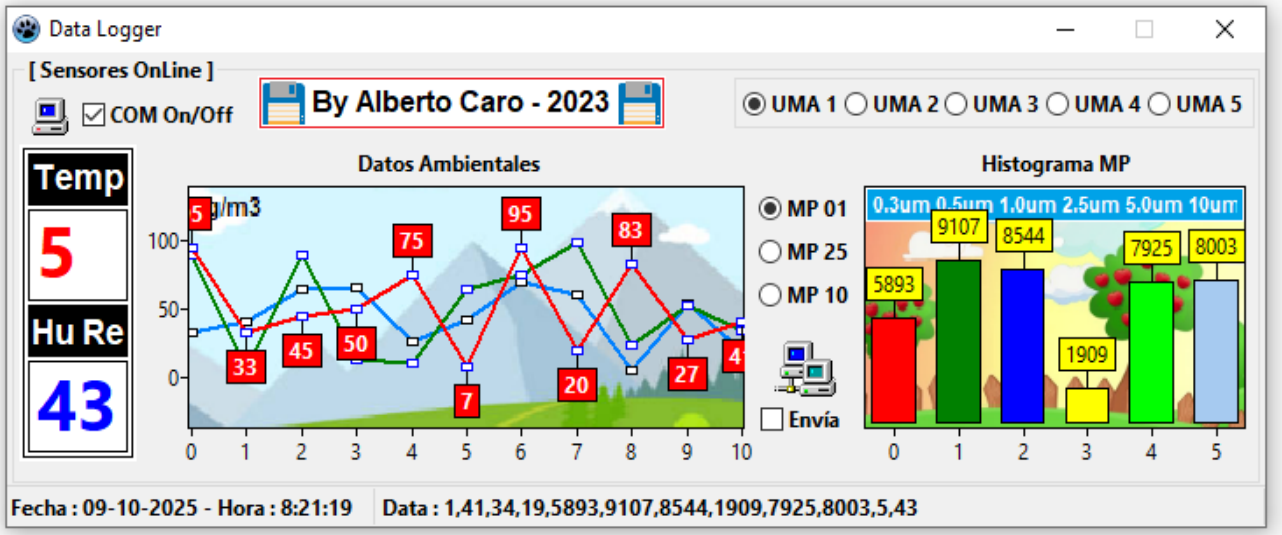
INFO1128

Proyecto #2 (Unidad II)

BY ALBERTO CARO

En el siguiente tutorial se muestra el uso de los componentes **Tchart, TLineSeries** y **TLazSerial** de **Lazarus Pascal** que implementan un **DataLogger** para el registro y visualización de contaminación del aire de las particulas **PM1.0, PM2.5 y PM10** um. También se utiliza el componente **TFPHttpClient** para enviar una estructura de **JSON** a un **Servidor de Datos** implementado con **Flask** de **Python**. La idea es utilizar este tutorial para desarrollar este interesante proyecto.

La Interfaz



En la interfaz principal se muestran los niveles de temperatura y humedad relativa del aire. Se aprecia también las **3** series de tiempo de los niveles de contaminación ambiental de **3** tipos de material particulado. En esta interfaz se puede seleccionar cuál **Unidad Medio Ambiental (UMA1-UMA5)** se desea sensar para ser graficada. Se resalta el uso del envío de los datos mediante un **Cliente HTTP** y el uso del puerto **Serial (COM)** para la recepción de los datos mediante el uso de **Virtual Serial Port Emulator (Connector)** desde **Serial Python**.

HTTPClient Lazarus

El objeto **TFPHTTPClient** en **Lazarus Pascal** es una clase que permite realizar solicitudes **HTTP** de manera sencilla, tanto para descargar contenido desde un **servidor web** como para enviar datos a través de métodos como **GET, POST, PUT, DELETE**, etc. Es parte de la biblioteca **fphttpclient**, que está incluida en **Free Pascal** y no depende de componentes externos como **Indy**.

Características principales de TFPHTTPClient:

1. **Soporte para múltiples métodos HTTP**: Permite realizar solicitudes **GET, POST, PUT, DELETE**, etc.
2. **Descarga de archivos**: Puede descargar archivos desde una **URL**.
3. **Envío de datos**: Permite enviar datos en el cuerpo de las solicitudes **POST o PUT**.
4. **Manejo de encabezados HTTP**: Puedes agregar **encabezados** personalizados a las solicitudes.
5. **Soporte para HTTPS**: Puede manejar conexiones seguras a través de **HTTPS** (si está habilitado el soporte **SSL**).
6. **Simple y ligero**: Es fácil de usar y no requiere dependencias externas.

Estructura de Datos

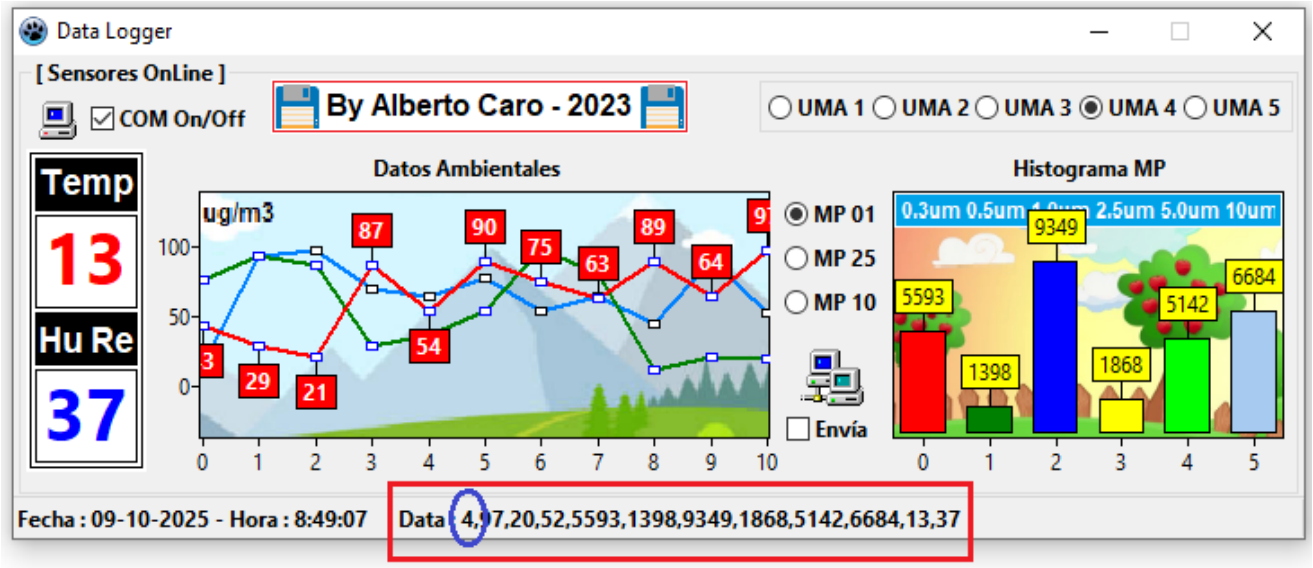
```
*main | LazSerial
      .   end;
      .
     75   Var
      .    MyForm : TMyForm;
      .    aMP01  : Array[0..11] of Integer;
      .    aMP25  : Array[0..11] of Integer;
      .    aMP10  : Array[0..11] of Integer;
     80    aHisto : Array[0..05] of Integer;
      .    aClr   : Array[0..5] of TColor;
      .    nID    : Byte;
      .    lFlask : Boolean;
```

Los arreglos **aMP01, aMp25, aMP10** se utilizan para almacenar de manera dinámica **12** muestras de los niveles de contaminación recibidos por comunicación serial enviadas por **Serial Python**. El arreglo **aHisto** se utiliza para almacenar el **Histograma (conteo)** de los **6** niveles de particulas de diámetros **0.3, 0.5, 1.0, 2.5, 5.0 y 10 um**. Y por último, el arreglo **aClr** que se utiliza para configurar los colores de las barras del histograma anterior.

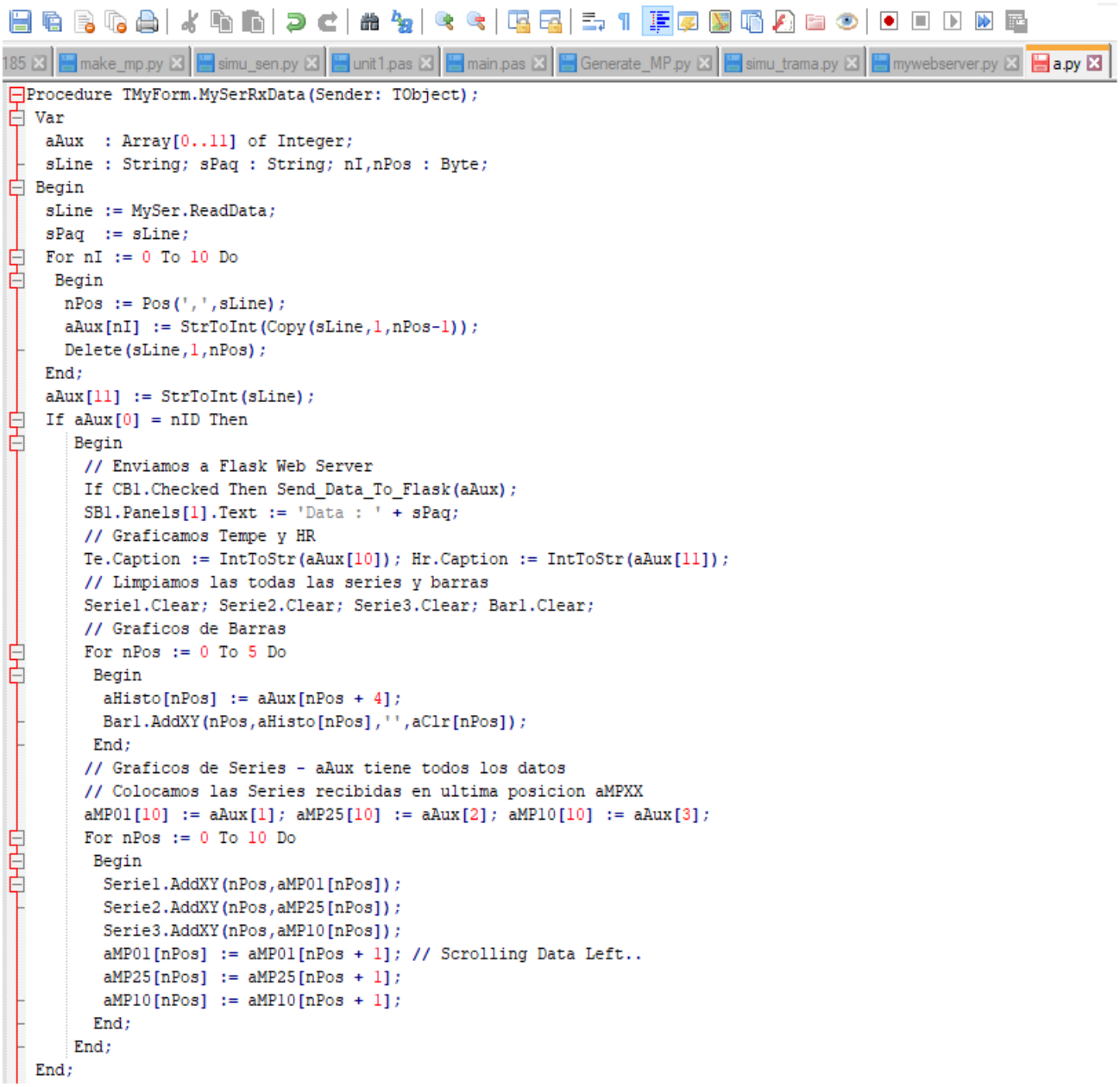
Limpieza de las Series

```
main | LazSerial
      .  Procedure TMyForm.Clear_Trama;
      .  Var
     160   nPos : Integer;
      .  Begin
      .   For nPos := 0 To 11 Do
      .     Begin
      .      aMP01[nPos] := 0; aMP25[nPos]:= 0; aMP10[nPos] := 0;
     165     Serie1.AddXY(nPos,aMP01[nPos]); Serie2.AddXY(nPos,aMP25[nPos]);
      .      Serie3.AddXY(nPos,aMP10[nPos]);
      .     End;
      .   For nPos := 0 To 5 Do
      .    Begin
     170     aHisto[nPos] := 0;
      .      Bar1.AddXY(nPos,aHisto[nPos],'A',aClr[nPos]);
      .    End
      .  End;
```

Cada vez que se reciben los **12** valores de cada una de las **3** series de niveles de contaminación se limpiar las series anteriores para preparar los datos antes de ser graficados. Se recuerda que cada trama comienza con el identificador de la **UMA** que envía los datos. En la siguiente imagen se muestra la trama de datos de la **UMA = 4**.



Evento Lectura Serial MySerRxData(Sender: Tobject)




```
Procedure TMyForm.MySerRxData(Sender: TObject);
 Var
  aAux  : Array[0..11] of Integer;
  sLine : String; sPaq : String; nI,nPos : Byte;
 Begin
  sLine := MySer.ReadData;
  sPaq  := sLine;
  For nI := 0 To 10 Do
   Begin
    nPos := Pos(',',sLine);
    aAux[nI] := StrToInt(Copy(sLine,1,nPos-1));
    Delete(sLine,1,nPos);
  End;
  aAux[11] := StrToInt(sLine);
  If aAux[0] = nID Then
     Begin
      // Enviamos a Flask Web Server
      If CB1.Checked Then Send_Data_To_Flask(aAux);
      SB1.Panels[1].Text := 'Data : ' + sPaq;
      // Graficamos Tempe y HR
      Te.Caption := IntToStr(aAux[10]); Hr.Caption := IntToStr(aAux[11]);
      // Limpiamos las todas las series y barras
      Serie1.Clear; Serie2.Clear; Serie3.Clear; Bar1.Clear;
      // Graficos de Barras
      For nPos := 0 To 5 Do
       Begin
        aHisto[nPos] := aAux[nPos + 4];
        Bar1.AddXY(nPos,aHisto[nPos],'',aClr[nPos]);
       End;
      // Graficos de Series - aAux tiene todos los datos
      // Colocamos las Series recibidas en ultima posicion aMPXX
      aMP01[10] := aAux[1]; aMP25[10] := aAux[2]; aMP10[10] := aAux[3];
      For nPos := 0 To 10 Do
       Begin
        Serie1.AddXY(nPos,aMP01[nPos]);
        Serie2.AddXY(nPos,aMP25[nPos]);
        Serie3.AddXY(nPos,aMP10[nPos]);
        aMP01[nPos] := aMP01[nPos + 1]; // Scrolling Data Left..
        aMP25[nPos] := aMP25[nPos + 1];
        aMP10[nPos] := aMP10[nPos + 1];
       End;
     End;
 End;
```

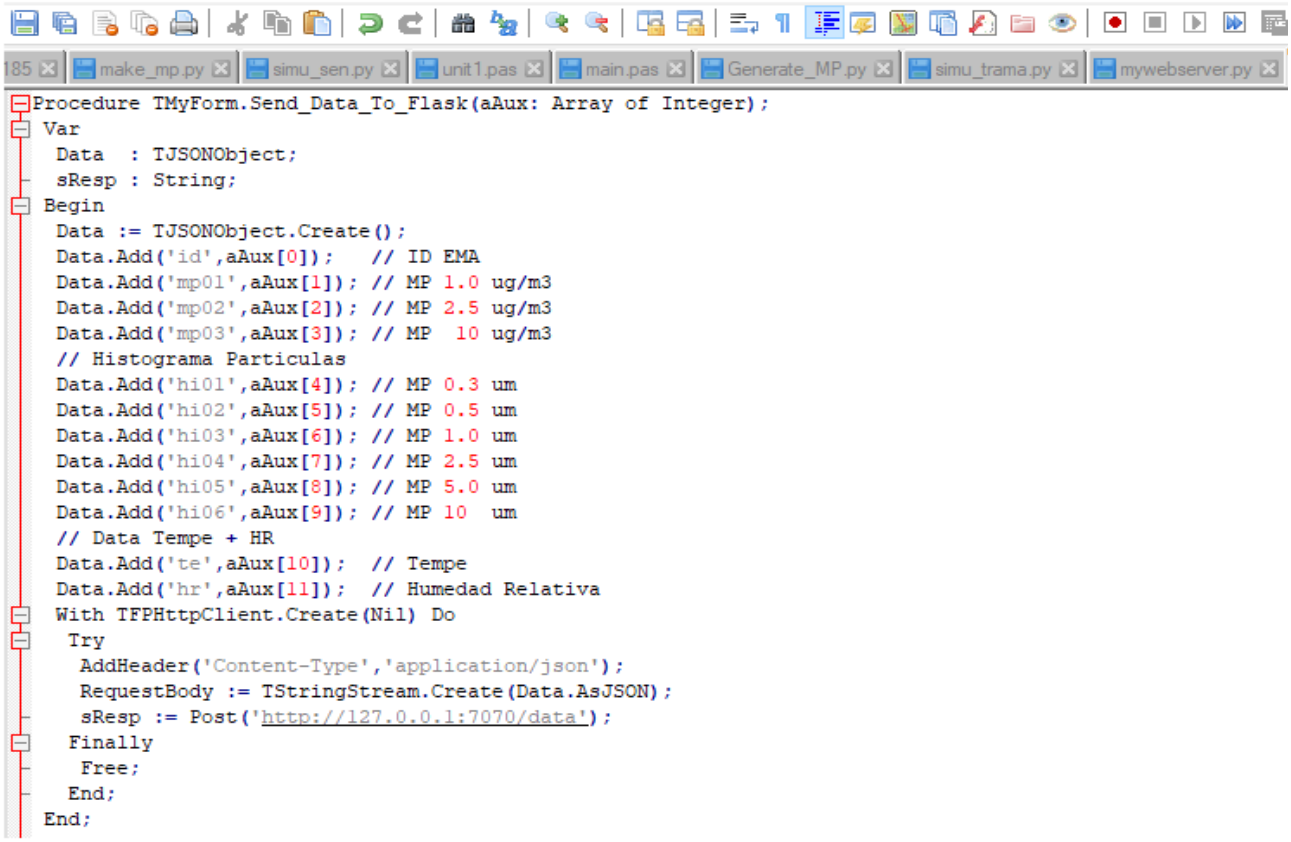
El evento **MySerRxData** del componente **LazSerial de Lazarus Pascal** se dispara cada vez que se reciben datos seriales por el puerto **COM2 a 9600-8N1**. Dentro de este evento, se separan los datos y se convierten a tipo **Integer**. Cada nivel de contaminación se coloca al final de cada una de las **3** series y también en el **Histograma**. Los datos se colocan al final de cada uno de sus arreglos, pues se tienen que desplazar de derecha a izquierda (**Scrolling Left**).

Cuando se activa el envío de los datos al **Server Flask** se llama al procedimiento **Send_Data_To_Flask** con todos los valores dentro del arreglo **aAUX** (**12 muestras**). En realidad, lo que senvía al **Server Flask** es una estrutura tipo **JSON** dentro del **Body** mediante **Post**.

¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato de intercambio de datos ligero y fácil de leer tanto para humanos como para máquinas. Es ampliamente utilizado en aplicaciones web y móviles para transmitir datos entre un servidor y un cliente, o entre diferentes componentes de un sistema.

Send_Data_To_Flask




```
Procedure TMyForm.Send_Data_To_Flask(aAux: Array of Integer);
Var
 Data  : TJSONObject;
 sResp : String;
Begin
 Data := TJSONObject.Create();
 Data.Add('id',aAux[0]);   // ID EMA
 Data.Add('mp01',aAux[1]); // MP 1.0 ug/m3
 Data.Add('mp02',aAux[2]); // MP 2.5 ug/m3
 Data.Add('mp03',aAux[3]); // MP  10 ug/m3
 // Histograma Particulas
 Data.Add('hi01',aAux[4]); // MP 0.3 um
 Data.Add('hi02',aAux[5]); // MP 0.5 um
 Data.Add('hi03',aAux[6]); // MP 1.0 um
 Data.Add('hi04',aAux[7]); // MP 2.5 um
 Data.Add('hi05',aAux[8]); // MP 5.0 um
 Data.Add('hi06',aAux[9]); // MP 10  um
 // Data Tempe + HR
 Data.Add('te',aAux[10]);  // Tempe
 Data.Add('hr',aAux[11]);  // Humedad Relativa
 With TFPHttpClient.Create(Nil) Do
  Try
   AddHeader('Content-Type','application/json');
   RequestBody := TStringStream.Create(Data.AsJSON);
   sResp := Post('http://127.0.0.1:7070/data');
  Finally
   Free;
  End;
End;
```

En el script anterior se aprecia la declaración del objeto **TJSONObject** el cual encapsula todo el manejo de los datos en formato **JSON**. En **Data(JSON)** se van cargando los pares **(key,Values)** que definen a un **JSON**. En **Data** se almacenan todas las variables críticas de esta simulación de contaminación ambiental. El primer elemento es '**id**' el cual identifica la **UMA** que envia los datos. Después, le siguen los demas datos de la trama.

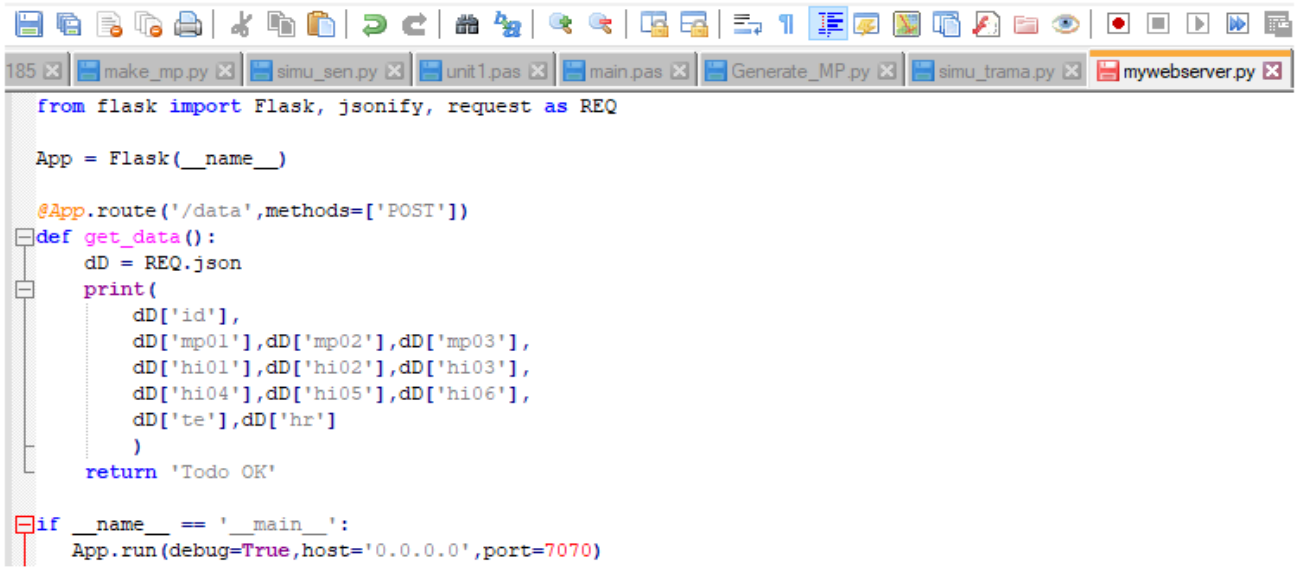
¿Qué papel juega Content-Type en una solicitud POST?

El encabezado **Content-Type** especifica el **tipo de contenido** de los datos que se está enviando en el cuerpo de la solicitud **POST**. Esto es importante porque el servidor necesita saber cómo interpretar los datos que recibe. Por ejemplo, si estás enviando datos en formato **JSON**, debes configurar el encabezado **Content-Type** como '**application/json**'. Esto le dice al servidor que el cuerpo de la solicitud es un objeto **JSON** y que debe procesarlo como tal. Analiza este ejemplo.




```
1  uses
2    fphttpclient, sysutils;
3  Var
4   Client: TFPHTTPClient; Response: String; JSONData: String;
5  Begin
6   Client := TFPHTTPClient.Create(nil);
7   Try
8    Client.AddHeader('Content-Type', 'application/json');
9    JSONData := '{"nombre": "Batman", "edad": 40}';
10   Response := Client.FormPost('http://ejemplo.com/api',JSONData);
11   WriteLn(Response);
12  Finally
13   Client.Free;
14 End;
15 End.
```

Server Flask




```python
from flask import Flask, jsonify, request as REQ

App = Flask(__name__)

@App.route('/data',methods=['POST'])
def get_data():
    dD = REQ.json
    print(
        dD['id'],
        dD['mp01'],dD['mp02'],dD['mp03'],
        dD['hi01'],dD['hi02'],dD['hi03'],
        dD['hi04'],dD['hi05'],dD['hi06'],
        dD['te'],dD['hr']
        )
    return 'Todo OK'

if __name__ == '__main__':
    App.run(debug=True,host='0.0.0.0',port=7070)
```

El **Server Flask** recibe los datos en formato **JSON** por método **POST** y los imprime en consola. Se podrían almacenar los datos en **SQLITE, Mariadb, PostGresSQL, FireBase, MongoDB, Oracle**, **DataSet Dinámico**, etc. Tu escojes.

Simu_Trama




```python
import serial as rs, time as ti, random as RA

def Make_Data(nId): # nId = Identificador de la UMA
    nEMA  = nId
    nMP01 = RA.randint(05,100)  # Material Particulado 1.0 um
    nMP25 = RA.randint(05,100)  # Material Particulado 2.5 um
    nMP10 = RA.randint(05,100)  # Material Particulado 10  um
    nHi03 = RA.randint(10,10000)# Histograma MP 0.3 um
    nHi05 = RA.randint(10,10000)# Histograma MP 0.5 um
    nHi01 = RA.randint(10,10000)# Histograma MP 1.0 um
    nHi25 = RA.randint(10,10000)# Histograma MP 2.5 um
    nHi50 = RA.randint(10,10000)# Histograma MP 5.0 um
    nHi10 = RA.randint(10,10000)# Histograma MP 10  um
    nTE   = RA.randint(05,30)   # Temperatura Celcius
    nHR   = RA.randint(20,70)   # Humedad Relativa Aire

    sLine  = str(nEMA ) + ',' + str(nMP01) + ',' + str(nMP25) + ',' + str(nMP10) + ','
    sLine += str(nHi03) + ',' + str(nHi05) + ',' + str(nHi01) + ','
    sLine += str(nHi25) + ',' + str(nHi50) + ',' + str(nHi10) + ','
    sLine += str(nTE  ) + ',' + str(nHR  )
    return (sLine)
MyCnx = rs.Serial('COM2'); MyCnx.baurate = 9600 ; nId = 1
while 1:
 sAux = Make_Data(nId)
 print(sAux)
 MyCnx.write(sAux)
 ti.sleep(0.1)
 nId += 1
 if nId ==6: nId = 1
MyCnx.close()
```

Asi se simula una trama de un sensor de material particulado por ejemplo **PMS7003ST** que envía los datos por **Serial.**

¡Facil y Bonito! La programación es muy entretenida, pues no solo te ayuda a ser más inteligente si no que además, aclara y agudiza tu pensamiento lógico y entrena tu cerebro para lograr mayores niveles de abstracción.

Se pide

1.- Implementar en **Lazarus Pascal** (Windows o Linux) el proyecto anterior.

2.- Por el lado del Server Flask, implemente una API REST que permita consultar por los registros almacenados en la base de datos (debera crear en SQLITE la base de datos y la Tabla respectiva). Los datos recibidos se deben almecenar con **DateTime**. Utilizando un navegador se debe visualizar las consultas en formato **JSON**. Además, se debe vizualizar una gráfica generada por **Matplotlib** en el lado del navegador.

4.- Instale **Virtual Serial Port Emulator** o algún **opensource** similar en **Linux**.

3.- Instale todos los componentes y librerias necesarias en **Lazarus Pascal**. Investigar.

4.- Fecha de entrega y defensa: **Viernes 30 de Octubre desde 9:00 AM.**

5.- Utiliza todos los códigos anteriores e iumplementa los cambios que consideres necesarios.

6.- Grupo de **2** Personas o Individual

7.- Sin informe impreso.

